

LangChain & Agentic AI – From Basics to Advanced demo- 8 Notes

static prompt vs Dynamic Prompt:

static prompt:

Explain Python in simple english

It is fixed and non changeable

Dynamic Prompt:

It is not fixed and changeable based user requirement

Explain { Topic} in simple english

{ Topic} --->input variable or placeholder

Final prompt = Template (Dynamic Prompt) + Variable values

PromptTemplate class:

It is used to create reusable text prompts with variables

```
from langchain_core.prompts import PromptTemplate

prompt_template = PromptTemplate.from_template("Explain
{topic} in Simple English")

#print(type(prompt_template))

#print(prompt_template)

#print(prompt_template.input_variables)


result = prompt_template.invoke(
    {
        "topic": "Python"
    }
)
```

```
)  
print(type(result))  
print(result)  
print(result.text)  
print(result.to_string())
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
prompt_template = PromptTemplate.from_template("Explain  
{topic} in Simple English")
```

```
topic = input('Enter Topic:')
result = prompt_template.invoke(
    {
        "topic":topic
    }
)
ai_message = llm.invoke(result)
print(ai_message.content)
```

User ---> PromptTemplate --->Final Prompt -->LLM ---
>AIMessage -->Response

=====

PromptTemplate invoke() ==>To format the prompt ie to
replace variables with original values

ChatModel invoke() ==>To submit prompt to llm and to get the
response

=====

Recipe Assistant

import os

from langchain_openai import ChatOpenAI

from langchain_core.prompts import PromptTemplate

llm = ChatOpenAI(

model="gpt-4o-mini",

api_key=os.getenv("OPENAI_API_KEY")

)

template = PromptTemplate.from_template(

"Give me a simple recipe for {dish}. Explain it step by step."

)

dish = input("Enter dish name: ")

```
prompt = template.invoke(  
    {  
        "dish": dish  
    }  
)  
response = llm.invoke(prompt)  
  
print(response.content)
```

=====

Topic Explainer

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",
```

```
api_key=os.getenv("OPENAI_API_KEY")  
  
)  
  
template = PromptTemplate.from_template(  
    """  
    Explain {topic} to a beginner.  
    """  
    )
```

Use:

- Very simple English
- 5 important points
- One real-life example

```
    """  
    )  
  
topic = input("Enter topic: ")  
  
prompt = template.invoke(  
    """  
    Explain {topic} to a beginner.  
    """  
    )
```

```
{  
    "topic": topic  
}  
)
```

```
response = llm.invoke(prompt)  
print(response.content)
```

```
=====
```

```
# Topic Explainer
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")
```

)

```
template = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} to a beginner.
```

```
Use:
```

- Very simple English
- 5 important points
- One real-life example

```
    """
```

```
)
```

```
topic = input("Enter topic: ")
```

```
prompt = template.invoke(
```

```
{
```

```
        "topic": topic
    }
)

response = llm.invoke(prompt)
print(response.content)
```